

18/9/26

ECSE101

Random Signals and Stochastic Processes

S. Kumarajay

Y. Mallikarjun Reddy

Probability

Albert Leon Garcia

20/7/26

Microcontroller and embedded systems.

ARM $\rightarrow$ Advanced Risc machine / microprocessor

PIC $\rightarrow$ Peripheral Interface controller (1993)

$\hookrightarrow$ By microchip technology.

Book $\rightarrow$ M. Mazidi
D. Causey

SP Das $\rightarrow$ NPTEL

8051 $\rightarrow$ 8 bit μ controller.

Intel 8051, 8031, 8096

National 6800HC 31

8051 consists of

- $\rightarrow$ 8 bit CPU
- $\rightarrow$ 4KB on chip program memory
- $\rightarrow$ 128 byte Data RAM
- $\rightarrow$ two 16 bit timer
- $\rightarrow$ four I/O ports
- $\rightarrow$ Full duplex serial port
- $\rightarrow$ On chip clock oscillator.

Memory organisation $\rightarrow$ Harvard architecture (separate units)

Program memory

Data memory.

Addressing mode → way of specifying operands.

PIC

RISC architecture

ROM = 2MB, RAM = 256 to 4096 byte

Data EEPROM, timer, ADC, USART, SPI for PC comm

I/O port → 16 to 72 pins.

★ 5 timers

Port A, B, C, D, E

0, 1, 2, 3, 4

0, 1, 3 → 8 bit

2, 4 → 16 bits

★ Working register → WREG

Status register

X	C	AC	Z	P	S	X	X
---	---	----	---	---	---	---	---

MPLAB IDE
↓
microchip pic → Made by microchip technology 1993

Memory part

368 bytes

4K * 14, 13 bit instructions.

A/D converter

ADCON 0

ADCON 1

Oscillator for timing generation

30/7/26

Criteria for selection

① handle the computational task effectively and efficiently

a) speed b) power consumption, c) No. of RAM/ROM

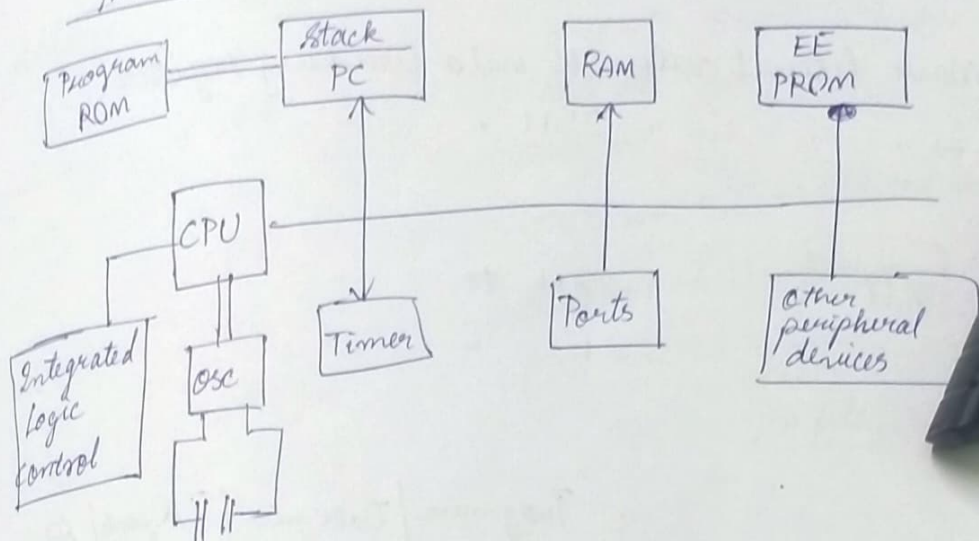
- 1) No. of I/O pins, e) Cost per unit
- 2) How easy to develop product around it, i.e., availability of compiler, debugger, emulator.
- 3) Adaptability & compatibility
- 10 XXX
12 XXX
14 XXX
16 XXX
17 XXX
18 XXX

Mechatronics

MEMS - micro electrical mechanical system

PIC $\Rightarrow$ 1989 - 1993 $\rightarrow$ 8bit MC

Micro architecture



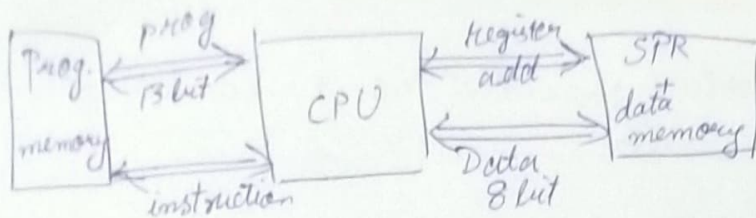
EPROM
UV $\leftarrow$ OTP
Flash
Mask version

PIC 16C432
 $\downarrow$
represents OTP

Flash - In this category the firmware parts are stored permanently and effectively.

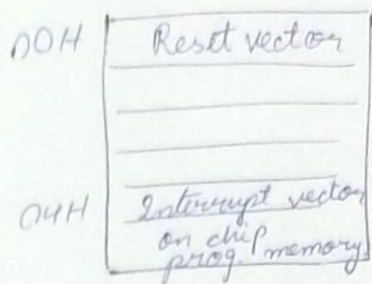
PIC F458

Mask version is the cheapest



Xtream@2047

CPU architecture



PIC 18 = 1.5X better than PIC 16

FPGA

ASIC - Application specific IC

MOVLWK → Move literal value K into working register.

MOVLW 25H → " " " 25H " " "

Load

ADD LW 28H → Add " " 28H to " "

ADD LW K → " " " K " " "

4/8/26

PIC

	Program mem	Data mem	I/O ports	ADC
① 16C74A → OTP EPROM	4K EPROM	32	8	10 bit
② 16F477 → Flash.	8K Flash	4096	8	4 channel

PIC controller → (20 to 40) MHz

40 pins.

1 instr. cycle → 4 clock cycle.

machine "
T cycle

Com[~] port
SPI
I²C

CPR and SFR

File register
Port A, B, C, D, E.

MOVWF → Move the wreg register value to file register.

MOVLW 55H ; WREG = ~~55H~~ 55H

MOVWF PORTC; Move (copy) the WREG to file register
(Port C = 55H)

MOVWF PORTD;
MOVWF PORTE;

↳ file register

ADDWF file reg, D destination bit.

↓
add contents of wreg to file.
if → D = 0 → WREG
D = 1 → file register.

Address	data
005	22
006	22
007	22
012	00
013	85
014	3F

MOVLW 22H , WREG = 22H
MOVWF 5H move wreg to 5H
MOVWF 6H
MOVWF 7H

ADDWF 5, 0 → Add the wreg = 22H
total (44H)

ADDWF 6, 0 → 66H in wreg.

ADDWF 7, 1 → 88H in file

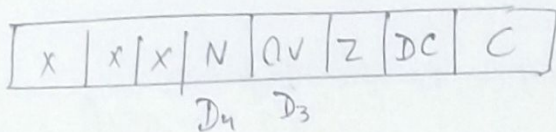
↓
value at add
005 to 22H
↓
(22H)

↓
and store in
wreg.

- ADD WF
- ADD WFC
- AND WF
- MOV FF
- OR WF
- SUB FW
- SUB B
- COMF
- DECF
- XORWF

INCF
RLCF

Status Register



Carry - it is set when there is carry out from D₇ bit in addⁿ / subⁿ

Digital carry - if there is carry generated from D₃ to D₄ during addⁿ / subⁿ, then this flag is set.

Zero - Result of ALU operation

Overflow - It is set when the result of signed operation is too large causing the higher order bits to overflow.

Negation - If D₇ bit of result is 0, then N=0; i.e., result is true.
 " " " " " " " 1, then N=1; " " " -ve

Q) Show the status of carry, digital carry, zero flag after the addⁿ.

MOV LW 38H

ADD LW 2FH; Add 2FH to WREG.

$$\begin{array}{r} 1 \\ 38H \\ 2FH \\ \hline 67H \end{array}$$

$$\begin{array}{r} 0011 \ 1000 \\ 0010 \ 1111 \\ \hline 0110 \ 0111 \end{array}$$

$$C = 0$$

$$DC = 1$$

$$Z = 0$$

$$\begin{array}{r} 1 \\ 9CH \\ 67H \\ \hline 100H \end{array}$$

$$C = 1$$

$$DC = 1$$

$$Z = 1$$

$$\begin{array}{r} 1001 \ 1100 \\ 0110 \ 0100 \\ \hline 1 \ 0000 \ 0000 \end{array}$$

8) $\begin{array}{r} 88 \text{ H} \\ 93 \text{ H} \\ \hline 2 \text{ B H} \end{array}$

$Z = 0$
 $C = 1$
 $DC = 0$

8/26 PIC File format and directives.

Data formats

- ① HEX
- ② Binary
- ③ Decimal
- ④ ASCII

Compiler → HLL to machine language

Assembler → Assembly language to machine language

Instructions and Pseudo instructions (directives)
 ↳ gives direction to assembler.

- ① EQU ~~used to do~~
- ② SET
- ③ ORG
- ④ END
- ⑤ LIST

① used to define a constant value or fixed address.

The following ins. uses Equate for counter constant and then the constant is used to store/load in WREG.

```
COUNT EQU 0x25
MOVLW COUNT; WREG = 25H
```

② used to define constant value or fixed address.

SET and EQU are identical

↳ values may be reassigned later

③ used to indicate the beginning of the address. Used for both code and data.

④ indicates the assembler end of the program file / source file. (.)

⑤ unique directive - indicates assembler about specific chip for which program should be assembled.

PIC ALP

Label Pneumonics operands Comments.

- ① We use a text editor to type in a program like MPLAB IDE which has text editor, assembler, linker, simulator and other details in one package. Some uses MS-DOS Edit or Notepad in windows.
- ② The 'asm' source file...
- ③ Program is written in diff formats, further interpreted stored at respective memory locations.

pg-87

Write a program to add the data 3 hex numbers, 2 binary numbers and store the result at location 15H.

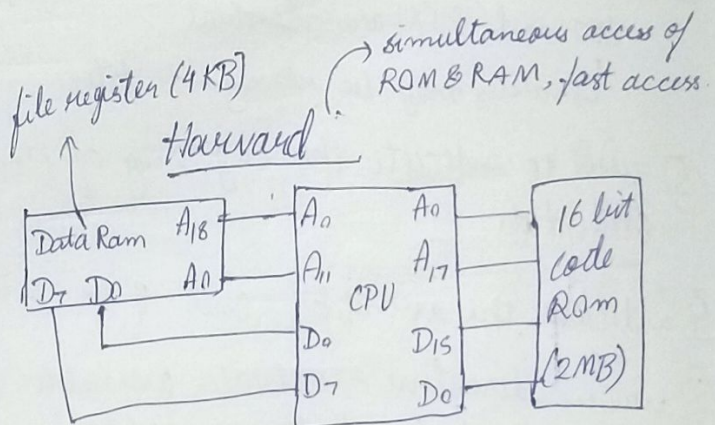
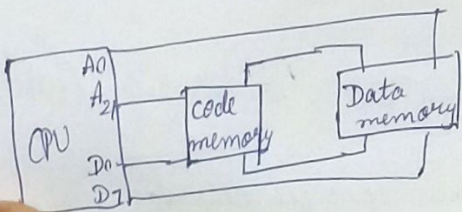
11/8/26

~~list~~ directives
list - indicate source & binary codes.
map - indicates memory layout of used and unused memory locations.
→ to identify the correct system design

Program counter → indicates / points to next instruction to be executed. Under the PC more memory locations CPU can access.

PIC 16 - 12 bit
PIC 18 - 21 bit

Van Neuman



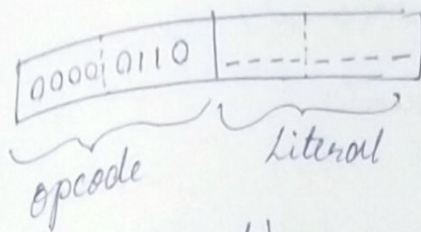
Instructions

Instructions

MOVLW, ADDLW, MOVWF, MOVF, GOTO

↓ ↓ ↪ 4 byte ←
2 byte inst. 2 byte inst. 2 byte

↪ 00-FFH



RISC (for speed)
frequency ↑ =

- ① RISC (for speed)
- ① Clock frequency $\uparrow \Rightarrow$ power consumption $\uparrow$
- ② Harvard architecture by increasing no. of system buses.
- ③ Change in internal architecture
 - $\rightarrow$ pipeline / parallel processing.
 - $\rightarrow$ 21-35 inst. increase $\rightarrow$ 75 inst.

MPLAB - assembler, linker, simulator.

Branching, call and time delay loop.

Looping in PIC - repeating a sequence of instructions or operation a certain number of times is called as loop. It is the most widely used programming technique.

DECFSZ instruction.

decrement file register & skip to 'Q'.

DECFSZ file register d $\rightarrow$ In this inst. the file register is decremented and if its content is zero then it will skip to next instruction by placing GOTO 'target' below it, we can create a loop.

BNZ $\rightarrow$ Branching if not zero.

Write a program to add value 3 to WREG, 10 times

COUNT EQU 0x25, use loc. 25H for counter

MOVLW d'10', WREG = 10 (decimal) " "

MOVWF COUNT, load the counter

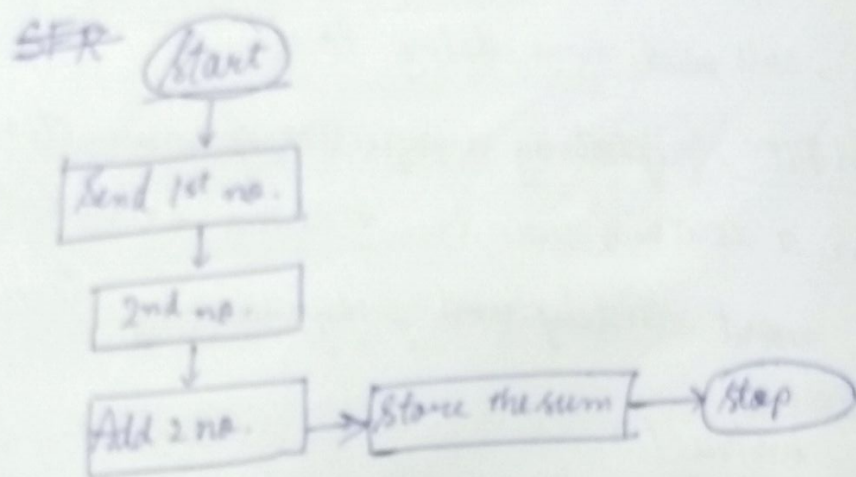
MOVLW 0; WREG = 0

ADD LW 3; add 03 to WREG

DECFSZ COUNTF; decrement counter $\rightarrow$ skip if count = 0

GOTO Again; Repeat until count becomes 0

MOVWF PortB; send sum to Port B SFR



Q) Write a program to add 6 to the WREG register 15 times
Use both inst., DECF and BNZ.

12/8/26

Nested loop

Q) Write a program to load the Port B SFR

R1 EQU 0x25

R2 EQU 0x26

COUNT-1 EQU d'10
 COUNT-2 EQU d'70
 MOVLW 0x55 ; WREG = 55H
 MOVWF PORTB ; PORTB = 55H
 MOVLW COUNT-1 ; WREG = 10, outer loop count value
 MOVWF R1 ; load 10 to loc 25H (outer loop count)
 MOVLW COUNT-2 ; WREG = 70 inner loop count
 MOVWF R2 ; load 70 into 26H

Write a program to load Port B SFR with value 55H and complement port B 700 times.

LOOP 2 COMF PORTB, F ; complement port B SFR
 DECF R2, F ; dec file reg. loc 26 (inner loop)
 BNZ LOP_2 ; repeat it 70 times
 DECF R1, F ; dec file reg loc 25 (outer loop)
 BNZ LOP_1 ; repeat it value (loop count) 10 times
 END

Conditional

Short jump 2 byte inst.
 → BNZ → no zero
 → BNC → no carry

Unconditional

→ GOTO 4 byte inst.

Call instructions

It is a controlled transfer instruction. It is used to call a sub routine → used to perform the task which is required to perform frequently.

long call → CALL
 4 byte

short call RCALL (relative)
 2 byte

32 bits $\rightarrow$ 12 opcode

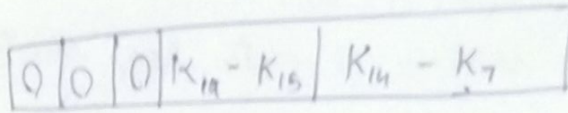
20 - Address of target subroutines

16 bits

5 "

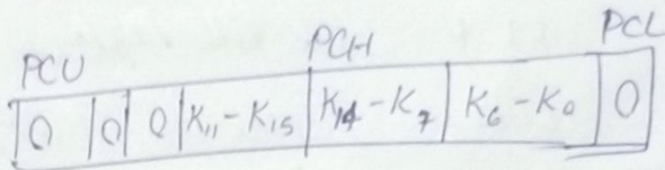
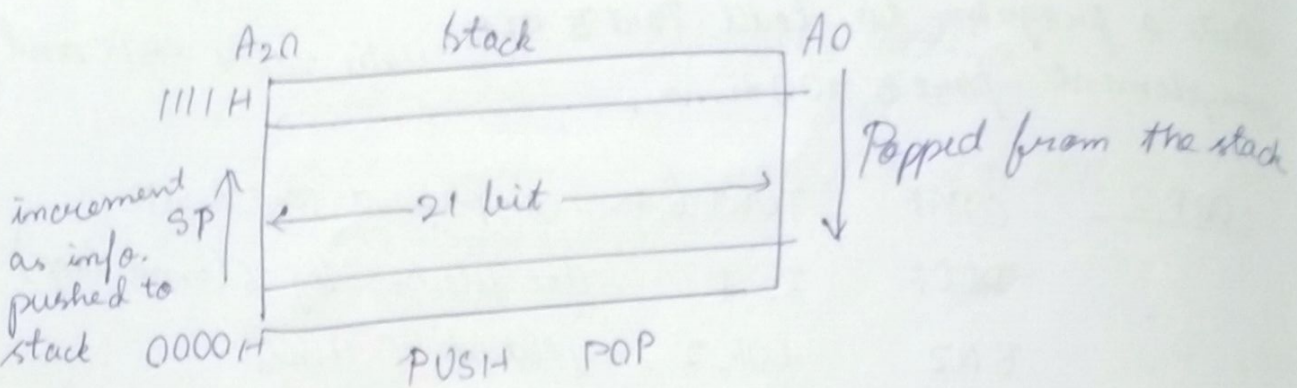
11 - "

2M
address
space



00000H - 1FFFFH

Stack and SP
 $\downarrow$ 21 bit
 $\rightarrow$ 8 bit register



Delay subroutine

Call Delay

PIC 18F458 $\rightarrow$ 33 I/O ^{pins} ~~pins~~ out of 40 pins

7 $\rightarrow$ OSC1, OSC2, ~~reset~~ ^{reset}, VDD, VSS, GND

Reset pin may
not be pushed in some
why?

PORTA, B, C, D, E
 $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$
 6/7 pin 8 8 8 $\rightarrow$ 3 pins

$PORT_x, TRIS_x, LAT_x \Rightarrow 3SFR$

$\downarrow$ $\downarrow$
 tristate Latch

13/3/26

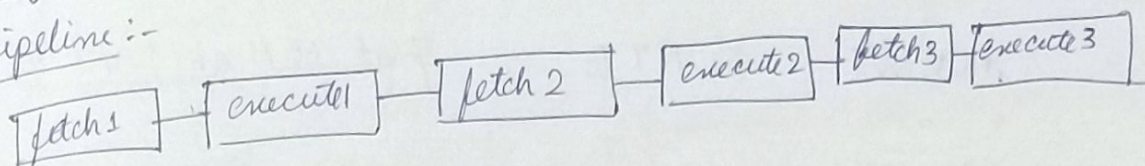
① Crystal frequency

② PIC design → no. of inst. present
↳ how much compatible with others.

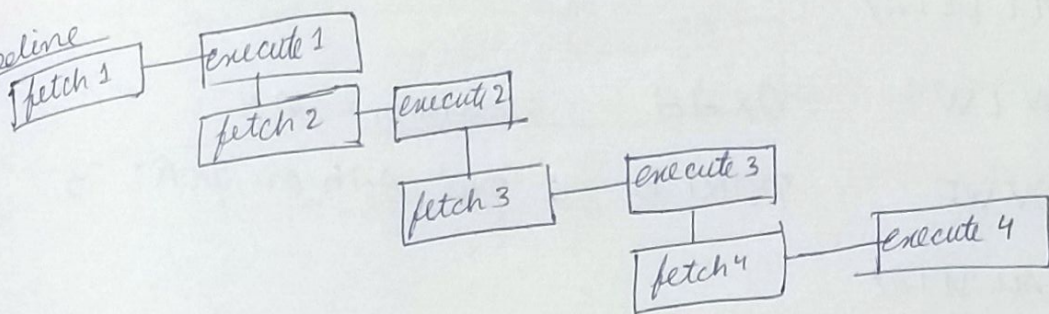
I/O Ports

Pipelining → fetch, decode, execute

Non pipeline :-



Pipeline



Instruction cycle in PIC

It takes certain amount of time for the CPU to execute an instruction. In PIC this time is known as instruction cycle/machine cycle. Since all the instructions in PIC 18 are of 2 byte or 4 byte so it takes not more than 1 or 2 instruction cycle to execute an instruction. In PIC 18, one instruction cycle is equal to 4 oscillator period = $\frac{1}{4} \text{th}$ crystal freq.

Length of inst. cycle depends on freq. of oscillator.

Q) Crystal freq. for 3 different PIC system are given as 4 MHz, 16 MHz, 20 MHz. Find period of inst. cycle in

each case.

1	4	5	10^{-6} s	$0.25 \times 10^{-6} \text{ s}$	$0.2 \times 10^{-6} \text{ s}$
			1 μs	0.25 μs	0.2 μs

TRIS_x → whether PORT will act as i/p or o/p

LAT_x → to store the instruction.

MOV LW 0x00 ; WREG = 00

MOV WF TRISB ; Make PORT B as an O/p port

L1 MOV LW 0x55 ; WREG = 55H

MOV WF PORTB ; Put 55H on PORT B pin

CALL DELAY

MOV LW 0xAA ; WREG = AAH

MOV WF PORTB ; Put AAH on PORT B

GOTO L1 CALL DELAY

Write prog. to toggle all 8 bits, using loop and call delay

Ports - All the ports in PIC18 have bidirectional and digital i/o capability. These ports are multiplexed along with the alternate functions for different peripheral devices and for the smooth functioning and execution of an instruction.

RA₀ - RA_{5,6}

RB₀ - RB₇

RC₀ - RC₇

RD₀ - RD₇

RE₀ - RE₂

ADD WFC

$$\begin{array}{r} 3C \quad E7 \\ + 3B \quad 8D \\ \hline 78 \quad 74 \end{array}$$

↑
C=1

40 = 7D
41 = EB
42 = CS
43 = 5B

L-byte EQU 0x6 ; Assign RAM location 6 to L-byte of sum.
H-byte EQU 0x7 ; " " " " " " " " " " " "

L-byte EQU 0x6 ; Assign RAM location 6 to L-byte of sum.
H-byte EQU 0x7 ; " " " " " " " " " " " "

H-lyte EOU 0x7;

movlw 0 ; clear WREG (WREG=0)

mov w1, 4-byte ; 4-byte = 0

ADDWF 0x40, W ; WREG = 0 + 7DH = 7DH, C = 0

BNC N-1 ; branch if $C=0$

INCF H-byte, F ; increment (new H-byte = 0)

N-1 ADDWF 0x41, W;

BNC N-2 ;

INCF H-lyte F;

N-2 ADD WF 0x42;

BNC N-3

INCF H-byte; C=1

N3 ADDWF 0x4B,W ; C = 0

$$\begin{array}{r} 2D \\ 5B \\ \hline 88H \end{array}$$

BNC N_L 4
INCF H-lyte F
N_L 4 MOVWF L-lyte

loc 6 = 88 H

DAW - Decimal adjust WREG → It works with an operand in a WREG register adding two BCD values using standard binary addition can yield illegal hex results.

$$29 + 01 = 2A$$

DAW inspects wreg along with the carry and DC flags in a status register. If a 4 bit nibble exceeds 9 DAW automatically adds 06 to adjust the byte back to BCD form.

19/3/26

use of DAW - After any inst.

① If the lower nibble is greater than 9 or DC=1 add 0110 to lower 4 bits.

② If the upper nibble is greater than 9 or C=1, add 0110 to upper 4 bits.

SUB LW K ; WREG = K - WREG

MOVLW 0x23 ; load 23H into WREG

SUB LW 0x3F ; WREG = 3F - WREG
↳ 1C

When C=1 , N=0 ⇒ Result = +ve

C=0 , N=1 ⇒ Result is -ve ⇒ 2's Complement.

8) Write a program to subtract 4C - 6E

MYREG EQU 0x20

MOVLW 0x4C ; load WREG, (WREG = 4CH)

MOVWF MYREG ; MYREG = 4CH

MOVLW 0x6E ; WREG = 6EH

SUBWF MYREG, W ; WREG = MYREG - WREG

BNN NEXT, if N=0 (C=1) jump to next target

NEGF WREG ; Take 2's comp.

NEXT MOVWF MYREG

```

0100 1100
1001 0010
-----
1101 1110

```

DE

write a program to subtract two 16 bit nos., the nos are 2762H - 1296H. loc 6 = 62, loc 7 = 27. Place the difference in file location 6 and 7. Loc 6 should have lower byte.

MOVLW 0x96 ; load the lower byte (WR = 96H)

SUBWF 0x6, F ; F = F - W, 62 - 96 = CCH | B = C = 1

MOVLW 0x12 ; load the high byte (WREG = 12H) | N =

SUBWFB 0x7, F ; F = F - W - b, subtract byte with borrow.

F = 27 - 12 - 1

```

  15
2762
1296
-----
14CC

```

```

-5  0101
    1111011

```

-81

1111011

128

-57

-128

128

80

-10000 0000

-34 - 128

20/8/26

-34 → CCH

-128 → 80H

MULLW K

DIVLW K

Compare and Rotate

3 inst^s.

① CMPFSGT → ~~compare~~ compare file register, skip if greater than
file reg > WREG

② CMPFSET → equal to

③ CMPFSLT → less than

8) Write a program to find the smaller number of the two values 27 and 54.
Place it in a file register location 0x20.

VAL-1 EQU D'27'

VAL-2 EQU D'54'

LREG EQU 0x20 ; location for smaller of two

MOVLW VAL-1 ; WREG = 27

MOVWF LREG ; LREG = 27

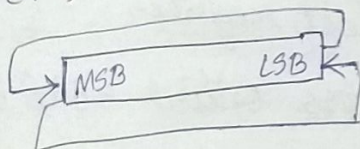
MOVLW VAL-2 ; WREG = 54

~~CMPSLT~~ LREG ; skip if LREG < WREG

MOVWF LREG ; place small value

Rotate

RRCF, RRCF, RLCF, RLNCF



PIC18F458

Addressing modes : Various ways of accessing data in memory location.

In PIC¹⁸ controller there are 4 addressing modes.

- ① Immediate
- ② Register indirect
- ③ Indexed RAM
- ④ Direct

In Immediate addressing mode the operand is the literal constant. That means, operand comes immediately after opcode.
when instr is assembled.

```
MOVLW 0x25
SUBLW D'62'
ANDLW AND WREG
B'0101101'
```

Immediate data called as literal in PIC 18.

Direct → operand data is in RAM memory location whose address is known.

```
MOVLW 0x56 ; WREG = 56H
MOVWF 0x40 ; copy wreg to file reg RAM loc 40H
MOVFF 0x40, 0x50 ; copy data from.
```

Register indirect → Register is used as a pointer to the data ~~not~~ RAM location. In PIC 18, 3 registers are used for this purpose. FSR 0, 1, 2
↓
file select register

FSR - 12 bit register that can access entire 4096 bytes that can of data RAM. LFSR is used to load RAM address

LFSR 0, 0x30

LFSR 1, 0x40

LFSR 2, 0x6F

Indexed RAM → data specifications are given in a programmable and memory access locations. ~~the~~

Advantages of C

267

- easier and less time consuming.
- easy to modify and update.
- code easily available in functional libraries
- portable to other MC without any modification.

25/8/26

Bit Data addressability in Data RAM

bit oriented instructions - RAM (4096 ^{bytes} ~~bits~~)
byte " " - RAM

Checksum data byte → Addⁿ of extra byte at end of series of data.

To use checksum byte, 2 rules:-

- ① Add the ~~sum~~ of bytes together and drop the carry.
- ② Take the 2's complement of the total sum, i.e., including checksum byte at the end of series of data.

Q) Assume that we have 4 ~~big~~ bytes of hexadecimal data 25H, 62H, 3FH, 52H.

- a) Find checksum byte
- b) Perform checksum operation to ensure data integrity.
- c) If the 2nd byte 62H is changed to 22H show how checksum byte will detect the error.

a) Ans. E8

c) 25
22
52
3F
E8

99
E8
1 8 1
3 F
E 0

1 C 0 H

87
3F
52
D9
3F

1 1 8 H

0001 1000
1110 1000
E 8

118
E8
200

Macros & Modules -

Macros are the applications in assembly language programming where a group of instructions used to perform the task repeatedly. Eg- moving data to RAM repeatedly.

It is not making any sense to rewrite the code everytime it is needed. Therefore to reduce the time and the possibility of error, the concept of macros is used.

name MACRO dummy1, dummy2, ...
 :::
 :::
 :::
 }
 ENDM
 ↓
 body of macro

MOVLW MACRO
 K } Body of Macro.
 MyReg }
 MOVWF
 ENDM

Advantage of using modules.

- 1) Failure of one module doesn't stop the program.
- 2) Each module can be written, debugged and tested individually.
- 3) Easier & less time consuming.
- 4) It can be linked with high level languages like C.

$$\begin{array}{r} 0001 \cancel{4} 1000 \\ 1110 1000 \\ E 8 \\ 118 \\ \underline{E8} \\ 200 \end{array}$$

- ### Advantage of using modules.
- 1) Failure of one module doesn't stop the program.
 - 2) Each module can be written, debugged and tested individually.
 - 3) Easier & less time consuming
 - 4) It can be linked with high level languages like C.

<u>C</u>		
<u>Data types</u>	<u>time delays in C.</u>	
unsigned char	→ 8 bit	0 - 255
char	→ 8 bit	-128 to 127 → D7 for sign
unsigned int	→ 16 bit	0 to 65,535
int	→ 16 bit	-32,768 - 32,767 → D15 for sign
short	→ 24 bit	
long	→ 32 bit	

Q) Write a C 18 program to send values 00-FFH to Port B.

```
#include <P18F458.h> // for TRISB and PORTB declaration
```

```
void main(void)
```

```
{ unsigned char z;
```

```
  TRISB = 0 // make port B an output
```

```
  for (z=0, z <= 255, z++)
```

```
    PORTB = z;
```

```
    while(1); // needed to keep running in hardware
```

```
}
```

Q) Write a program to send Hex values for ASCII characters of 0,1,2,3,4,5,A,B,C,D to port B.

1/9/26

```
Ans) #include <P18F458.h>
```

```
void main(void)
```

```
{ unsigned char my-num[] = "012345ABCDEF";
```

```
  unsigned char z;
```

```
  TRISB = 0;
```

```
  for (z=0; z < 10, z++)
```

```
    PORTB = my-num[z];
```

```
  while(1)
```

```
}
```

Q) Write a program to toggle all bits of Port B continuously.

```
#include <P18F458.h>
```

```
void main(void)
```

```
{ TRIS B = 0;
```

```
  for(;;){
```

```
    PORTPort B = 0x55;
```

```
    PORT B = 0xAA; }
```

```
}
```

Count no. b/w -4 to 4

Q) Write a C18 prog. to toggle all bits of PORT B 50000 times

```
{ unsigned int z;
```

```
  TRIS B = 0;
```

```
  for(z=0, z <= 50000; z++)
```

```
  { PORT B = 0x55;
```

```
    PORT B = 0xAA;
```

```
  } while(1);
```

for 1,00,000 use long data type.

Time delay

① Using a simple for loop

② Using PIC18 timer

Q) LED's are connected to the bits in PORT B and PORT C. Write

C18 program that shows the count 0000 - FFFF

0000 0000 1111 1111 on LEDs

Q) Write a program to toggle all bits of Port B continuously.

```
#include <P18F458.h>
```

```
void main(void)
```

```
{ TRIS B = 0;
```

```
  for(;;){
```

```
    PORT Port B = 0x55;
```

```
    PORT B = 0xAA; }
```

```
}
```

Count no. b/w -4 to 4

Q) Write a C18 prog. to toggle all bits of PORT B 50000 times.

```
{ unsigned int z;
```

```
  TRIS B = 0;
```

```
  for(z=0, z <= 50000; z++)
```

```
  { PORT B = 0x55;
```

```
    PORT B = 0xAA;
```

```
  } while(1);
```

for 1,00,000 use long data type.

Time delay

① Using a simple for loop

② Using PIC18 timer

Q) LED's are connected to the bits in PORT B and PORT C. Write a

C18 program that shows the count 0000 - FFFF

0000 0000 '1111 1111 on LEDs

include < PIC18F458.h >

define LED PORTC

void main (void)

{ TRIS B = 0

TRIS C = 0

PORT B = 00

LED = 0

for (;;)

{ PORT B++

LED++

}

}

I/O programming in PIC18

① Byte addressability

② Bit " "

PORTx bit RB7

Timers

2-5 timers

T0, 1, 2, 3, 4

PIC 16 - 3 timers

PIC 18 - 5 " "

PIC 16 → Timer 0, Timer 1, Timer 2

↓
8 bit

↓
16 bit

↓
8 bit

3 timers are used in PIC 16 →

Function

① To create time delay.

② To act as counter.

Every timer needs a clock pulse to tick. The clock source may be internal or external. If we use internal ~~source~~ clock source

$\frac{1}{4}$ th of the frequency of crystal oscillator is fed to the timer
Fosc/4

Thus it is used for time delay generation. By choosing the external clock we ~~feed~~ choose pulses through one of the PIC 18 pins. Thus it will act as a counter.

D₇-D₁₅ → TMROH

D₀-D₆ → TMRQL

Timer control register → TCON

Timer clock source → TOCS

↳ tell whether
int. or ext. ~~the~~ timer

2/9/26

Timers

Timer control register = 8 bits

TMRQ ON → D₇ → It tells us the timer on and off control bit.
1 → ON, 0 → OFF, stop
start.

TOB bit → 1 → 8 bit
0 → 16 bit

TOCS - timer clock source select bit. 1 → external
0 → Fosc/4 RA4[clk] Pin

TOSE - Timer source edge select bit 1 → H to L
0 → L to H

PSA - Prescaler assignment bit

↳ to maintain resolution
" accuracy
" time delay
" overflow

$$\frac{16\text{MHz}}{8} = 2\text{MHz}$$

1 → timer clock bypasses
prescaler

0 → timer clock will include
prescaler

in powers of 2

these are timer prescaler selector
 $\left(\frac{\text{clock}}{4}\right) / 2, 4, 8, 16, 32, 256$

8) Write a C-18 program to toggle all the bits of PORT B continuously with some delay. Use timer 0, 16 bit mode and no prescaler option to generate the delay.

```
#include <P18F458.h>
void T0Delay(void);
void main(void);
{ TRIS B = 0
  while (1)
  { PORT B = 0X55;
    T0Delay();
    PORT B = 0XAA;
    T0Delay();
```

```
void T0Delay {
    T0CON = 0X08 // timer 0, 16 bit mode, no prescaler.
    TMROH = 0X35 // load TH0
    TMROL = 0X00 // load TLO
    T0CON bit TMROON = 1 // turn on T0
    while INTCON bits TMR0IF = 0 // wait for TFO to roll over
    T0CON bit TMROON = 0 // turn off T0
    INTCON bits TMR0IF = 0
    / Clear TF = 0
```

9) Write a C18 program to toggle only the port B 4 bit continuously every 50 ms. Timer 0, 16 bit mode, Prescaler = 1:4 to create delay.

```
#include <P18F458.h>
void T0Delay(void);
```

```
void main(void);
```

```
{ TRISB = 0
```

```
  while(1)
```

```
{ PORTB = 0x55;
```

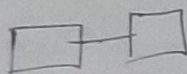
```
  T0Delay();
```

```
  PORTB = 0xAA;
```

```
  T0Delay();
```

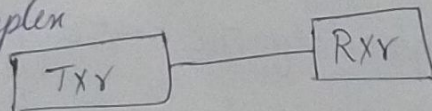
PIC 18 Serial port programming.

Series

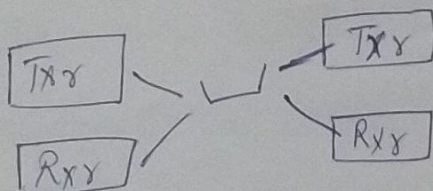


chances of loss of data is less ^{why?} data sent bit by bit
cost effective

Simplex

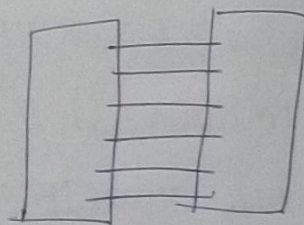


Half duplex

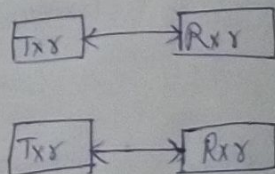


Parallel

Printer, Hard disk.



Duplex



```
void main(void);
```

```
{ TRISB = 0
```

```
  while(1)
```

```
    { PORTB = 0x55;
```

```
      T0Delay();
```

```
      PORTB = 0xAA;
```

```
      T0Delay();
```

PIC 18 Serial port programming.

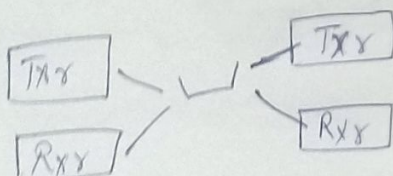
Series

chances of loss of data is less why? data sent bit by bit
cost effective

Simpler

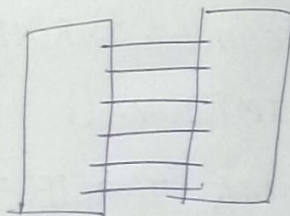


Half duplex

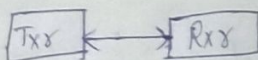
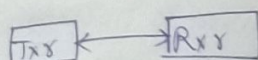


Parallel

Printer, Hard disk.



Duplex



3/9/26

Q) Find the timer clock frequency for the following PIC18 MC system whose crystal frequency is a) 10MHz, b) 16MHz, and time period.
No prescaler

a) $\frac{10}{4} = 2.5 \text{ MHz}$, $T = \frac{10^{-6}}{2.5} = 0.4 \mu\text{s}$

b) $\frac{16}{4} = 4 \text{ MHz}$, $T = \frac{10^{-6}}{4} = 0.25 \mu\text{s}$

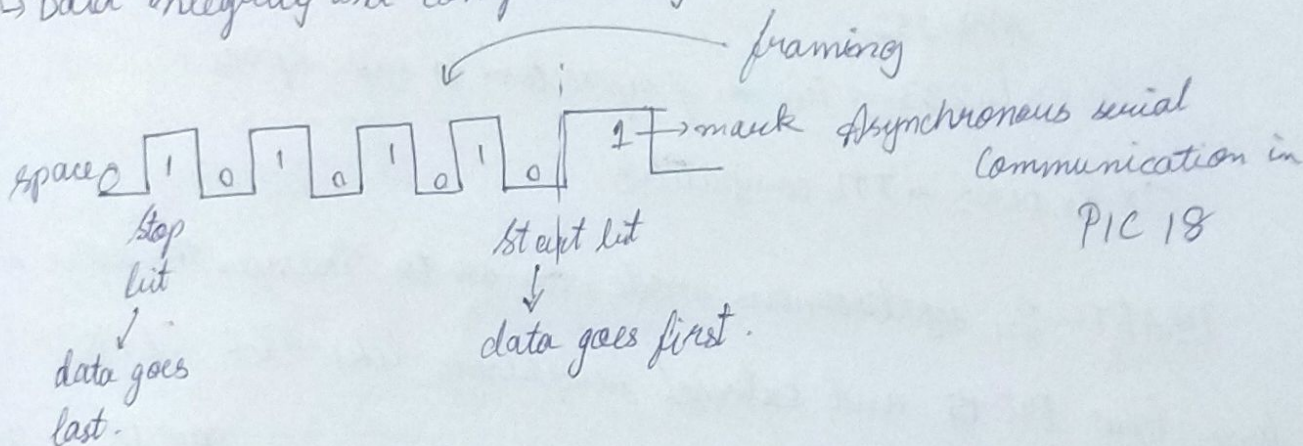
c) $\frac{4}{4} = 1 \text{ MHz}$ $\Rightarrow T = 10^{-6} = 1 \mu\text{s}$

Asynchronous serial communication & Data framing

Character oriented transmission

Synchronous $\rightarrow$ block oriented transmission.

$\rightarrow$ Data integrity and compatibility.



bps = bits per second

baud rate = no. of signals which change state during transition.

RS-232 , RS-232 A/B/C

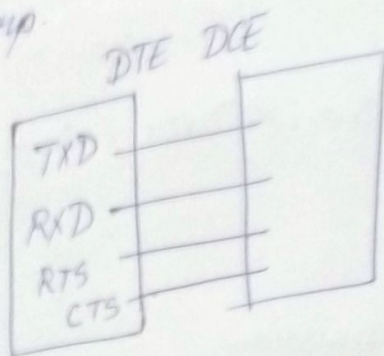
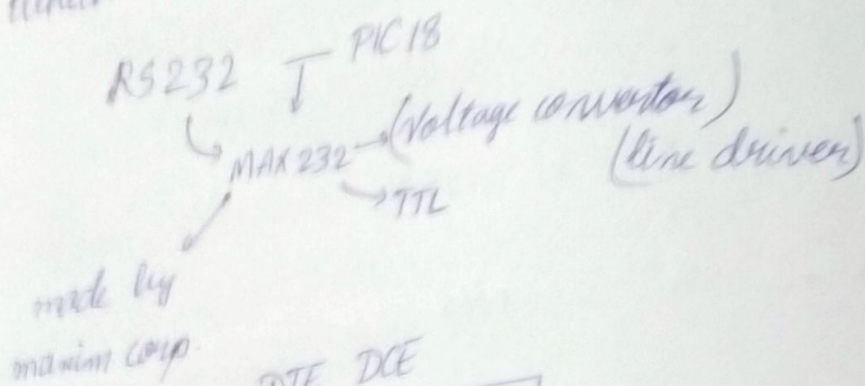
$\rightarrow$ Recommended standard

DB-25 connector $\rightarrow$ P-plug (male)

$\rightarrow$ data base S-socket (female)

EIA $\rightarrow$ Electronics industry of Association.

RS232 - 19 A
 cation equipments, a interface standard was developed
 by EIA in 1960s. It is a serial I/O interfacing standard
 which is used by the all PIC 18 MC system.



RS-232 $\downarrow$ PIC 18 $\rightarrow$ Com Port programming
 MAX-232

$\hookrightarrow$ 233 $\rightarrow$ less no. of capacitors to save space.

Tx, Rx pins $\rightarrow$ TTL compatible

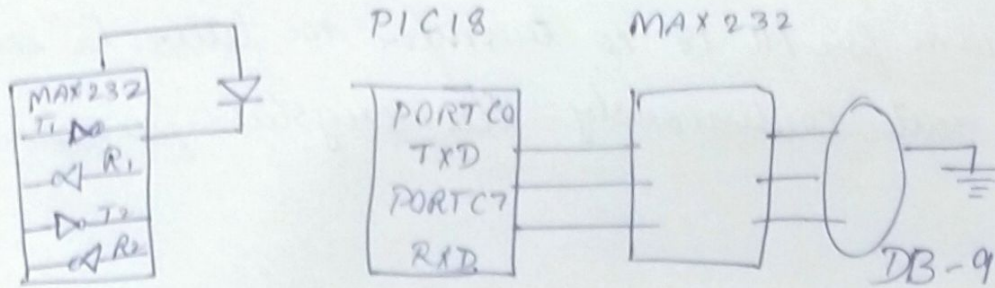
USART - In synchronous mode, ~~as use to~~ transfer the data is done b/w PIC 18 and external peripherals like ADC, EEPROMs.

The asynchronous mode will connect the ~~for~~ PIC 18 with IBM PC serial port for the purpose of full duplex serial data transfer.

In PIC 18 MC there are 6 $\rightarrow$ all 8 bit registers associated with USART.

① SPBRGR $\rightarrow$ ~~TXREG~~ - serial port baud rate generator

- ② TXREG - Transfer register
- ③ RCREG - Receive register
- ④ TXSTA - Transmit status & control register
- ⑤ RCSTA - Receive status and control register
- ⑥ PIR - Peripheral interrupt request register



RS232

40pin DIP PIC18

Write a program to transmit the message YES serially at 9600 baud, 8bit data and 1 stop bit, then do it forever.

```

MOVLW    B'00100000'
MOVWF    TXSTA
MOVLW    D'15'
MOVWF    SPBRG
BCF      TRISC, TX
BSF      RCSTA, SPEN
OVER
    MOVLW    A 'Y'
    CALL    TRANS
    MOVLW    A 'E'
    CALL    TRANS
    MOVLW    A 'S'
    CALL    TRANS

```

BRA OVER

TRANS —

SI BTFSS PIR, TXFF

BRA SI

MOVWF TXREG

RETURN

Q) Write a program for PIC 18 to transfer the letter 'G' serially at 9600 baud rate continuously. Use crystal frequency of 10 MHz

$$\begin{array}{r}
 88 \text{ H} \\
 93 \text{ H} \\
 \hline
 1 \ 2 \text{ B H}
 \end{array}
 \quad
 \begin{array}{l}
 Z = 0 \\
 C = 1 \\
 DC = 0
 \end{array}$$

50/8/26

PIC File format and direction

Data formats

- ① HEX
- ② Binary
- ③ Decimal
- ④ ASCII

Compiler → HLL to machine language

Assembler → Assembly language to machine language

Instructions and Pseudo instructions (directives)

↳ gives direction to assembler

- ① EQU - used to define
- ② SET
- ③ ORG
- ④ END
- ⑤ LIST

① used to define a constant value or fixed address.

The following ins. uses Equate for constant and then the constant is used to store/load in WREG.

```

COUNT EQU 0x25
MOVLW COUNT; WREG = 25H

```

② used to define constant value or fixed address.

SET and EQU are identical

↳ values may be re-assigned later

③ used to indicate the beginning of the address. Used for both code and data.

④ indicates the assembler end of the program file / source file - (.asm)

⑤ unique directive - indicates assembler about specific chip for which program should be assembled.

Write a program to add value 3 to WREG 10 times

COUNT EQU 0x25, use loc. 25H for counter

MOVLW d'10'; WREG = 10(decimal) " "

MOVWF COUNT; load the counter

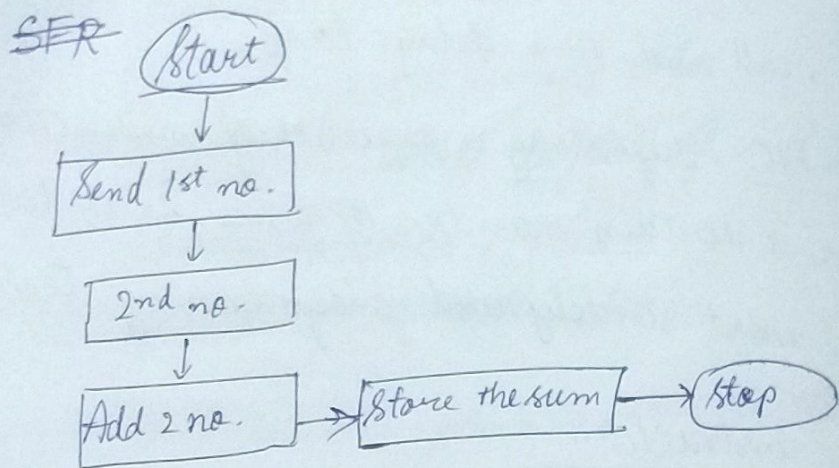
MOVLW 0; WREG = 0

ADD LW 3; add 03 to WREG

DECFSZ COUNTF; decrement counter $\rightarrow$ skip if count = 0

GOTO Again; Repeat until count becomes 0.

MOVWF PortB; send sum to Port B SFR



Q) Write a program to add 6 to the WREG register 15 times
Use both inst., DECF and BNZ.

12/8/26 Nested loop

Q) Write a program to load the Port B SFR

R1 EQU 0x25

R2 EQU 0x26